

Session

648bd223

Agent pipeline

1. **Planner** — refines your question
2. **Document Reader** — searches local guide
3. **Web Enricher** — Google Serper live search
4. **Synthesizer** — combines sources (Groq)
5. **Report Writer** — formats report (Gemini)

Try these

What are the biggest LLM security risks?

How does RAG reduce hallucinations?

What is LLM-as-Judge evaluation?

Best practices for deploying LLM agents?

Document Intelligence Agent

LangGraph · LangSmith · Groq · Gemini · Google Serper



what is THE LLM PRODUCTION STACK



Overview of a Typical Large Language Model (LLM) Production Stack

A typical LLM production stack involves several components and technologies, including data preprocessing, model training, deployment, and maintenance. The following sections provide a detailed analysis of each stage.

Data Preprocessing

The data preprocessing stage is critical in preparing the data for model training. This stage involves:

- **Ingestion Pipeline:** Converts raw documents (PDFs, web pages, databases) into vector embeddings stored in a vector database [1](#).
- **Chunk Size:** The chunk size is critical, with typical sizes ranging from 256 to 1024 tokens and a 10-20% overlap to avoid cutting sentences at boundaries [1](#).
- **Data Pipelines:** Tools like Apache Airflow and Prefect manage ingestion, preprocessing, and transformation of data from raw sources [2](#).

Model Training

The model training stage involves:

- **Neural Network Layers:** Embedding, attention, and feedforward layers work together to process the input text [2](#).
- **Machine Learning Pipeline:** This pipeline is responsible for training the LLM using the preprocessed data.

Deployment

The deployment stage involves:

- **Gateway Layer:** Routes and provides resilience for the LLM application [1](#).
- **Guardrails Layer:** Enforces safety and compliance policies on every request and response, using tools like NVIDIA NeMo Guardrails and Colang [1](#).
- **Evaluation Layer:** Measures the quality of the LLM application [1](#).
- **Observability Layer:** Monitors and debugs the LLM application [1](#).
- **Helm Chart:** Installs the standard vLLM production stack [2](#).
- **MLflow:** Provides tools and APIs for managing the model lifecycle, making it easy to deploy and administer LLMs in a production setting [2](#).

Maintenance

The maintenance stage involves:

- **Monitoring Frameworks:** Monitors the LLM application for performance and issues [2](#).
- **Governance Controls:** Ensures compliance with safety and compliance policies [2](#).
- **Optimization Mechanisms:** Optimizes the LLM application for better performance [2](#).

In conclusion, a typical LLM production stack involves a range of components and technologies, from data preprocessing to deployment and maintenance. By understanding these components and technologies, developers can build and deploy effective LLM applications.

References:

Footnotes

1. DOCUMENT KNOWLEDGE [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#)
2. WEB SEARCH RESULTS [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#) [↩⁷](#)

Pipeline details



what are LLM SECURITY THREATS



Security Threats and Vulnerabilities in Large Language Models (LLMs)

Large Language Models (LLMs) are powerful tools with various applications, but they also introduce specific security threats and vulnerabilities. According to the OWASP Top 10 for LLM Applications, deploying language models poses critical security risks.

Types of Security Threats

The following are some of the specific security threats and vulnerabilities associated with LLMs:

- **Data Poisoning:** This occurs when an adversary injects malicious data into the training phase of an LLM, compromising its performance and reliability [1].
- **Model Inversion Attacks:** These attacks involve reverse-engineering an LLM to extract sensitive information, such as intellectual property or personal data [2].

- **Adversarial Attacks:** These attacks exploit weaknesses in an LLM to manipulate its output, potentially leading to fraudulent or malicious activities [3].
- **Prompt Injection:** This involves crafting malicious prompts to manipulate an LLM's output, potentially leading to security breaches or unauthorized access to data [4].
- **Backdoor Attacks:** These attacks involve inserting malicious code or data into an LLM, allowing an adversary to access or manipulate the model's output [5].
- **Jailbreaking:** This involves exploiting vulnerabilities in an LLM to bypass security controls or access restricted functionality [6].

Defense Techniques

To mitigate these security threats, it is essential to implement defense techniques, such as:

- **Validating LLM Output:** Treating LLM output as untrusted user input and validating it before passing it to downstream systems [7].
- **Sanitizing Input:** Sanitizing user input to prevent malicious data from being injected into an LLM [8].
- **Implementing Access Controls:** Implementing access controls to restrict access to sensitive data and functionality [9].
- **Monitoring and Auditing:** Monitoring and auditing LLM activity to detect and respond to potential security threats [10].

Conclusion

LLMs introduce specific security threats and vulnerabilities, including data poisoning, model inversion attacks, and adversarial attacks. To mitigate these risks, it is essential to implement defense techniques, such as validating LLM output, sanitizing input, and implementing access controls. By understanding these security threats and vulnerabilities, organizations can take proactive steps to protect their LLMs and prevent potential security breaches.

References:

- [1] Jaffal, N. (2025). The subcategories include backdoor attacks, data poisoning, prompt injection, and jailbreaking, each paired with the corresponding defense techniques.
- [2] Models can be reverse-engineered or stolen, leading to intellectual property and privacy breaches.
- [3] Attackers can use LLMs to analyze known vulnerabilities and craft malicious prompts.
- [4] This study comprehensively examines two critical security vulnerabilities in LLMs: prompt injection and data poisoning attacks.
- [5] The subcategories include backdoor attacks, data poisoning, prompt injection, and jailbreaking, each paired with the corresponding defense techniques.
- [6] Jailbreaking: This involves exploiting vulnerabilities in an LLM to bypass security controls or access restricted functionality.
- [7] LLM02 — Insecure Output Handling occurs when LLM responses are passed to downstream systems without validation.
- [8] Sanitizing user input to prevent malicious data from being injected into an LLM.
- [9] Implementing access controls to restrict access to sensitive data and functionality.
- [10] Monitoring and auditing LLM activity to detect and respond to potential security threats.

Pipeline details



what is RAG ARCHITECTURE PATTERNS



Introduction to RAG Architecture Patterns

RAG (Retro-Active Git) architecture patterns are designed to support event sourcing and microservices integration in software development. According to the document, RAG grounds LLM responses in actual documents, reducing hallucinations [1](#). The web search results suggest that RAG is an advanced method used to enhance the performance of large language models (LLMs) by integrating external knowledge sources into their response [2](#).

Key Characteristics of RAG Architecture Patterns

The key characteristics of RAG architecture patterns include:

- **Retrieval-Augmented Generation:** RAG grounds LLM responses in actual documents, reducing hallucinations [1](#).
- **Vector Databases:** Store embeddings and support approximate nearest-neighbor search, with popular options including FAISS, Qdrant, and Weaviate [1](#).
- **ReAct Pattern:** The most common agent architecture, where the LLM alternates between Thought, Action, and Observation [1](#).
- **Event Sourcing:** RAG patterns define how events are identified, captured, and produced based on specific triggers or conditions [2](#).
- **Microservices Integration:** RAG allows developers to decompose their system into manageable, independently deployable, and scalable units [2](#).

Applications of RAG Architecture Patterns

The applications of RAG architecture patterns include:

- **Enhancing LLM Performance:** RAG integrates external knowledge sources into LLM responses, improving their performance [2](#).
- **Event-Driven Systems:** RAG patterns enable the detection of events and production of responses based on specific triggers or conditions [2](#).

- **Microservices Architecture:** RAG supports the decomposition of systems into manageable, independently deployable, and scalable units².

Support for Event Sourcing and Microservices Integration

RAG architecture patterns support event sourcing and microservices integration by:

- **Providing a framework for event-driven systems:** RAG patterns define how events are identified, captured, and produced².
- **Enabling microservices integration:** RAG allows developers to decompose their system into manageable, independently deployable, and scalable units².
- **Integrating external knowledge sources:** RAG enhances LLM performance by integrating external knowledge sources into their responses².

In conclusion, RAG architecture patterns are designed to support event sourcing and microservices integration in software development. They provide a framework for event-driven systems, enable microservices integration, and integrate external knowledge sources into LLM responses.

References:

Footnotes

1. Document knowledge, Chapter 3: RAG Architecture Patterns^{↩ ↩² ↩³ ↩⁴}
2. Web search results, various sources^{↩ ↩² ↩³ ↩⁴ ↩⁵ ↩⁶ ↩⁷ ↩⁸ ↩⁹}

Pipeline details

▼



what is RAG ARCHITECTURE PATTERNS ?



Introduction to RAG Architecture Patterns

RAG (Retro-Active Git) architecture patterns, also known as Retrieval-Augmented Generation, play a significant role in software development by grounding Large Language Model (LLM) responses in actual documents, thereby reducing hallucinations [1](#). This approach is particularly useful in environments where information changes frequently or where precision is critical, such as software development and legal domains [2](#).

Key Characteristics of RAG

The key characteristics of a production RAG system include:

- **Five components:** A typical RAG system consists of five components, although the specifics of these components are not detailed in the provided document [1](#).
- **Retrieval and Generation:** RAG combines the strengths of retrieval and generation models, allowing for more accurate and up-to-date responses [2](#).
- **Real-time Data:** RAG uses real-time data from documents or databases before generating responses, ensuring that the information provided is current [2](#).
- **Hybrid AI Framework:** RAG is a hybrid AI framework that bolsters LLMs by combining them with external, up-to-date data [2](#).

Applications of RAG

The applications of RAG architecture patterns are diverse and include:

- **Software Development:** RAG is especially useful in software development, where precision and the ability to handle frequently changing information are crucial [2](#).

- **Legal Domains:** RAG's precision and ability to handle complex, frequently updated information make it an ideal choice for legal applications [2](#).
- **Generative AI Applications:** RAG allows developers to provide the latest research, statistics, or news to generative models, making it a valuable tool for building generative AI applications [2](#).
- **Business Implementation:** RAG makes it easier for businesses to implement AI by reducing the need for extensive training or custom models [2](#).

RAG Architectures

There are several RAG architectures, each designed to handle specific tasks, including:

- **Simple RAG:** Ideal for simple tasks, this architecture retrieves relevant chunks and feeds them into the LLM for direct generation [2](#).
- **Multi-Retriever RAG:** This architecture is more complex and allows for the handling of more intricate tasks [2](#).
- **Other Architectures:** There are at least 8 popular RAG architectures, each with its strengths and applications [2](#).

In conclusion, RAG architecture patterns are a powerful tool in software development, offering a range of benefits, including precision, the ability to handle frequently changing information, and ease of implementation. By understanding the key characteristics and applications of RAG, developers can leverage this technology to build more effective and efficient AI systems.

References:

Footnotes

1. CHAPTER 3: RAG ARCHITECTURE PATTERNS [↩](#) [↩²](#)

2. Web Search Results on RAG Architecture Patterns [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#) [↩⁷](#) [↩⁸](#) [↩⁹](#) [↩¹⁰](#) [↩¹¹](#)

Pipeline details



what is LLM EVALUATION



Evaluating Large Language Models (LLMs): Methodologies and Metrics

Evaluating the performance and effectiveness of Large Language Models (LLMs) is crucial in natural language processing tasks. This analysis combines document knowledge and web search results to provide a comprehensive overview of standard methodologies and metrics used for LLM evaluation.

Standard Methodologies

The evaluation of LLMs typically involves standardized benchmarks, such as:

- GLUE (General Language Understanding Evaluation) [1](#)
- RAG metrics, including faithfulness and answer relevancy [2](#)
- Agent metrics and LLM-as-a-judge [2](#)

Additionally, Token Cost Optimization strategies can be employed to improve the efficiency of LLMs, including:

- Semantic caching (avoiding LLM calls for repeated questions)
- Prompt compression (removing redundant context before sending)
- Model routing (using cheap models for simple tasks and expensive models for complex tasks)
- Output caching (caching expensive computations like embeddings) [3](#)

Standard Metrics

Common metrics used to evaluate LLMs include:

- **Perplexity:** measures the uncertainty of the model [4](#)
- **Accuracy:** measures the proportion of correct predictions [4](#)
- **F1-score:** measures the balance between precision and recall [4](#)
- **ROUGE score:** measures the quality of generated text [4](#)
- **BLEU score:** measures the quality of generated text [4](#)
- **METEOR score:** measures the quality of generated text [4](#)
- **Question answering metrics:** measures the model's ability to answer questions [4](#)
- **Faithfulness:** measures the model's ability to generate faithful responses [2](#)
- **Answer relevancy:** measures the model's ability to generate relevant responses [2](#)

Evaluation Framework

A comprehensive evaluation framework for LLMs should include:

- **Performance Assessment:** evaluating the model's performance on various tasks [5](#)
- **Knowledge and Capability Evaluation:** evaluating the model's knowledge and capabilities [5](#)
- **Alignment and Safety Evaluation:** evaluating the model's alignment with human values and safety [5](#)
- **Use of Evaluation Metrics:** using metrics like BLEU, ROUGE, factuality, and toxicity to evaluate the model's performance [5](#)

In conclusion, evaluating LLMs requires a comprehensive approach that includes standardized benchmarks, Token Cost Optimization strategies, and a range of metrics to assess the model's performance, knowledge, and safety.

References:

Footnotes

1. S Joshi · 2025 · Cited by 19 [↩](#)
2. This review paper provides a comprehensive survey of the key metrics and methodologies employed to evaluate LLMs. [↩](#) [↩²](#) [↩³](#) [↩⁴](#)
3. DOCUMENT KNOWLEDGE: Token Cost Optimization strategies [↩](#)
4. Web Search Results: LLM evaluation metrics [↩](#) [↩²](#) [↩³](#) [↩⁴](#) [↩⁵](#) [↩⁶](#) [↩⁷](#)
5. Web Search Results: LLM evaluation framework [↩](#) [↩²](#) [↩³](#) [↩⁴](#)

Pipeline details



what is OBSERVABILITY



Definition and Scope of Observability

Observability in software systems refers to the ability to measure a system's current state based on the data it generates, such as logs, metrics, and traces [1]. It allows teams to understand the internal state or condition of a complex system based solely on knowledge of its external outputs [2]. In essence, observability is the ability to examine and understand the system state [3].

Relation to Monitoring, Logging, and Tracing

Observability is closely related to monitoring, logging, and tracing in distributed architectures. It connects metrics, logs, and traces to provide a unified view of system behavior, enabling faster

troubleshooting and debugging [4]. Monitoring systems are an essential part of observability, as they help detect errors and anomalies [5]. Logging provides detailed information about system events, while tracing allows for distributed tracing through collecting and aggregating traces, logs, and metrics [6].

Key Components of Observability

The three main components of observability are:

- **Logs:** provide detailed information about system events
- **Metrics:** offer quantitative data about system performance
- **Traces:** enable distributed tracing and provide a detailed view of system interactions

Benefits of Observability

Observability provides several benefits, including:

- Faster troubleshooting and debugging
- Improved system understanding and visibility
- Enhanced monitoring and logging capabilities
- Unified view of system behavior

In summary, observability is a critical aspect of software systems, allowing teams to understand the internal state of a system by examining its external outputs. It is closely related to monitoring, logging, and tracing, and provides a unified view of system behavior, enabling faster troubleshooting and debugging.

References: [1] Web Search Results: Observability is the ability to measure a system's current state based on the data it generates, recorded as logs, metrics, and traces. [2] Web Search Results: Observability is the ability to understand the internal state or condition of a complex system based solely on knowledge of its external outputs. [3] Web Search Results: Observability is the ability to examine and understand the system state. [4] Web Search Results: Observability gives

teams a unified view of system behavior by connecting metrics, logs, and traces, enabling faster troubleshooting and debugging. [5] Web Search Results: Monitoring systems are an important part of most systems. [6] Web Search Results: Observability allows for distributed tracing through collecting and aggregating traces, logs, and metrics.

Pipeline details ^

```
▼ {  
  "refined_question" :  
    "What is the definition and scope of observability in software  
    systems, and how does it relate to monitoring, logging, and tracing  
    in distributed architectures?"  
  "pipeline" :  
    "planner → document_reader → web_enricher → synthesizer →  
    report_writer"  
  "doc_sections_found" : 3  
  "session_id" : "648bd223"  
}
```

Ask anything about LLM production...

